

Practice 13 详解答案

A. 单项选择题

1. **B**: 构造函数在对象创建时负责初始化对象。
2. **C**: 构造函数名与类名相同, 写作 `Note()`, 且没有返回类型。
3. **B**: 对象创建时会自动调用匹配的构造函数。
4. **C**: 析构函数名是在类名前加 `~`, 即 `~Note()`。
5. **B**: 局部对象离开其作用域时自动调用析构函数。
6. **C**: `Note n("Meeting");` 调用带一个字符串参数的构造函数。
7. **A**: 重载构造函数使对象可用不同参数方式初始化。
8. **A**: `RAII` 让对象在构造时取得资源、在析构时自动释放资源。
9. **C**: 构造函数没有返回类型。
10. **A**: 对象的创建、使用和销毁构成了明确的生命周期。

B. 填空题

1. 构造函数
2. 析构函数
3. 返回
4. `~`
5. 重载

C. 判断题

1. **对**: 一个类可声明多个参数列表不同的构造函数。
2. **对**: 构造函数可按参数数量或类型重载。
3. **错**: 一个类只能有一个析构函数, 且它没有参数。
4. **对**: 未声明构造函数时, 编译器通常会隐式生成默认构造函数。
5. **对**: 例如 `class Note` 的构造函数名为 `Note`。
6. **对**: 局部对象离开作用域时, 析构函数自动执行。
7. **错**: 析构函数不支持重载, 也不能带参数。
8. **对**: 构造函数在对象开始使用前初始化成员。
9. **对**: `RAII` 将资源的获取和释放绑定到对象生命周期。

D. 代码阅读

题面中出现的 `classNote`、`intmain()`、`stringtitle` 均应按 C++ 语法改为 `class Note`、`int main()`、`string title` 后再分析。

2. 析构函数打印什么

```
class Note
{
public:
    ~Note()
    {
        cout << "Destroyed";
    }
};

int main()
{
    Note n;
}
```

输出：**Destroyed**。n 在 main 结束时离开作用域，析构函数自动执行。

3. 调用哪个构造函数

```
Note n("Study");
```

调用带参数构造函数 `Note(string title)`，输出：**Parameterized**。实参 "Study" 匹配一个字符串参数。

4. 代码问题

```
void Note()
{
}
```

这是一个名为 `Note` 的普通成员函数，不是构造函数，因为它写了返回类型 `void`。构造函数不能写返回类型：

```
Note()  
{  
}
```

5. 输出顺序

构造 `n` 时输出 `A`；离开 `main` 时析构 `n`，输出 `B`。因此总输出为：**AB**。

E. 简答题

1. 构造函数如何提升可靠性

构造函数使对象一创建就完成必要初始化。例如笔记对象可保证标题不是未初始化状态，文件对象可保证打开状态被正确建立。这样调用者不能忘记手动初始化，类的每个对象一开始就更容易处于有效状态。

2. 析构函数的目的

析构函数在对象生命周期结束时自动运行，用来清理对象拥有的资源，例如关闭文件、释放动态内存、解除锁。对于只包含 `string` 等自动管理资源的类，通常不必手写析构函数；成员对象会自动完成自己的清理。

3. 为什么 RAII 是良好设计

RAII 将资源管理交给对象生命周期：构造函数取得资源，析构函数释放资源。无论函数正常返回、提前 `return`，还是因异常离开作用域，局部对象仍会析构，因此资源不容易泄漏。

F. 编程题

1. 第一个构造函数

```
#include <iostream>
using namespace std;

class Student
{
public:
    Student()
    {
        cout << "Student object created.\n";
    }
};

int main()
{
    Student student;
}
```

创建 student 时自动调用 Student()，输出 Student object created.。

2. 带参数构造函数

```
#include <iostream>
#include <string>
using namespace std;

class Book
{
private:
    string title;
    string author;

public:
    Book(const string& bookTitle, const string& bookAuthor)
        : title(bookTitle), author(bookAuthor)
    {
    }

    void display() const
    {
        cout << "Title: " << title << '\n';
        cout << "Author: " << author << '\n';
    }
};

int main()
{
    Book book("Clean Code", "Robert C. Martin");
    book.display();
}
```

: title(bookTitle), author(bookAuthor) 是成员初始化列表，成员在构造函数主体开始前就被初始化。

3. 构造函数重载

```
#include <iostream>
#include <string>
using namespace std;

class Note
{
private:
    string title;

public:
    Note() : title("Untitled")
    {
    }

    Note(const string& noteTitle) : title(noteTitle)
    {
    }

    void display() const
    {
        cout << "Title: " << title << '\n';
    }
};

int main()
{
    Note first;
    Note second("Study Constructors");

    first.display();
    second.display();
}
```

输出标题依次为 Untitled 和 Study Constructors 。

4. 析构函数演示

```
#include <iostream>
using namespace std;

class Logger
{
public:
    Logger()
    {
        cout << "Program Started\n";
    }

    ~Logger()
    {
        cout << "Program Finished\n";
    }
};

void runProgram()
{
    Logger logger;
    cout << "Working...\n";
}

int main()
{
    runProgram();
}
```

输出顺序是：

```
Program Started
Working...
Program Finished
```

logger 的作用域是 runProgram()；该函数结束时对象析构。

5. 带构造函数的 CLI Notes（挑战）

```
#include <iostream>
#include <string>
using namespace std;

class Note
{
private:
    string title;
    string content;
    string tag;

public:
    Note()
        : title("Untitled"), content(""), tag("General")
    {
    }

    Note(const string& noteTitle, const string& noteContent, const string& noteTag)
        : title(noteTitle), content(noteContent), tag(noteTag)
    {
    }

    void display() const
    {
        cout << "==== NOTE =====\n";
        cout << "Title: " << title << '\n';
        cout << "Content: " << content << '\n';
        cout << "Tag: " << tag << '\n';
        cout << "=====\\n";
    }
};

int main()
{
    Note defaultNote;
    Note studyNote("C++ Constructors", "Read about RAII.", "Study");
    Note meetingNote("Team Meeting", "Discuss the project plan.", "Work");

    defaultNote.display();
    studyNote.display();
    meetingNote.display();
}
```


每个 `Note` 对象创建时都会调用一个构造函数。无参数时使用默认值，提供三个参数时使用传入的标题、内容和标签。

G. 设计与思考挑战

设计 B 更受推荐，因为 `title` 被设为 `private`，对象只能在创建时通过构造函数获得初始标题。对象不会经历“已经创建但标题尚未设置”的临时无效状态；调用代码也不依赖内部成员名称。后续可以在构造函数内校验标题、改变存储方式，而不必修改外部代码。设计 A 允许任何代码随时直接改写 `title`，封装性较弱。